

MODELOS Y SIMULACIÓN

INFORME DE TRABAJO PRÁCTICO

Joaquín Lopez Verdini
Eric Josias Rodriguez

Descripción del problema a simular

Centro de atención con servidores en serie

Los clientes llegan a un centro de atención para realizar trámites.
Todos los clientes deben atravesar dos etapas consecutivas:

1. Recepción, donde se verifica la documentación inicial.
2. Validación, donde se procesa el trámite y se emite la resolución correspondiente.

Cada etapa cuenta con un único servidor y una cola de espera de capacidad infinita. Los clientes que finalizan la primera etapa ingresan inmediatamente a la cola de la segunda.

Las llegadas al sistema siguen un proceso de Poisson no homogéneo con intensidad

$$\lambda(t) = 8 + 4 \sin(\pi t/12)$$

donde t se mide en horas.

Los tiempos de servicio en Recepción son variables aleatorias exponenciales con media de 4 minutos, mientras que los tiempos de servicio en Validación son exponenciales con media de 6 minutos.

Los tiempos de servicio son independientes entre sí e independientes del proceso de llegadas

Actividad 1

- a) **Escribir un programa para simular el sistema mediante eventos discretos.**

Para simular el sistema, tomaremos como referencia el algoritmo descrito en la bibliografía Ross, *Sheldon M. Simulation, 5ª edición* para atención al cliente por dos servidores en serie (tandem).

Este algoritmo simula un sistema de colas secuencial o tándem en el que cada cliente llega según un proceso de Poisson no homogéneo, recibe servicio primero en el servidor 1 y luego en el servidor 2 antes de partir. Los tiempos de servicio tienen distribuciones G_1 y G_2 , respectivamente.

Las variables que se mantienen durante la simulación son:

- Tiempo t : tiempo simulado transcurrido.
- Variables de estado ($n1, n2$): número de clientes en servidor 1 (incluyendo en cola y en servicio) y número en servidor 2.
- Contador NA: número total de llegadas hasta tiempo t .
- Contador ND: número total de salidas hasta tiempo t .
- Variables de salida: $A1[n] = t$ = tiempo de llegada del cliente n , $A2[n] = t$ = tiempo de llegada del cliente n a servidor 2, $D[n] = t$ = tiempo de salida del cliente n .
- Lista de eventos: $EL = (tA, t1, t2)$ donde tA = próximo evento de llegada, $t1$ = fin de servicio en servidor 1 (o ∞), $t2$ = fin de servicio en servidor 2 (o ∞).

Inicialización de las variables:

Establecer $t = NA = ND = 0$

Establecer $n1 = 0, n2 = 0$

Generar $T0$ (tiempo hasta próxima llegada)

Establecer $tA = T0, t1 = \infty, t2 = \infty$

Casos de actualización:

Mientras $tA < \infty$ o $t1 < \infty$ o $t2 < \infty$ hacer

// Encontrar el próximo evento

eventoProximo = mínimo($tA, t1, t2$)

// CASO 1: $tA = \text{mínimo}(tA, t1, t2)$ [Próxima llegada]

si eventoProximo = tA entonces

$t = tA$

$NA = NA + 1$

$n1 = n1 + 1$

Generar Tt (tiempo hasta próxima llegada)

$tA = t + Tt$

si $n1 = 1$ entonces

Generar $Y1$ (tiempo de servicio en servidor 1)

$t1 = t + Y1$

fin si

Registrar $A1(NA) = t$

fin si

// CASO 2: $t1 < tA$ y $t1 < t2$ [Fin de servicio en servidor 1]

sino si eventoProximo = $t1$ entonces

$t = t1$

$n1 = n1 - 1$

si $n1 = 0$ entonces

$t1 = \infty$

sino

Generar $Y1$

```

        t1 = t + Y1
    fin si
    n2 = n2 + 1
    si n2 = 1 entonces
        Generar Y2 (tiempo de servicio en servidor 2)
        t2 = t + Y2
    fin si
    numCliente = NA - n1 // Número del cliente que sale de servidor 1
    Registrar A2(numCliente) = t
    fin si
// CASO 3: t2 < tA y t2 < t1 [Fin de servicio en servidor 2]
    sino
        t = t2
        ND = ND + 1
        n2 = n2 - 1
        si n2 = 0 entonces
            t2 = ∞
        sino
            Generar Y2
            t2 = t + Y2
        fin si
        numCliente = NA - n1 - n2 // Número del cliente que sale del
sistema
        Registrar D(numCliente) = t
    fin si
    actualizarEstadísticas()
fin mientras

```

Implementación en python

```
def sim_servidores_serie(lamda_p, lamda_s1, lamda_s2, N):
    t, na, nd, n1, n2 = 0, 0, 0, 0, 0
    ta = gen_va_poisson(f, lamda_p, t)
    t1 = math.inf
    t2 = math.inf
    numCliente = 0
    lista_llegadas_sistema = []
    lista_llegadas_s2 = []
    lista_salidas_sistema = []
    tiempo_total_acumulado = 0

    # Fase de Actualizacion
    while ta < math.inf or t1 < math.inf or t2 < math.inf:

        # Determinamos cual será el evento mas proximo a suceder
        evento_proximo = min(ta, t1, t2)

        # Caso en el que un cliente llegó al sistema
        if evento_proximo == ta:
            t = ta
            na = na+1
            n1 = n1+1
            ta = t + gen_va_poisson(f, lamda_p, t)
            if na == N:
                ta = math.inf
            if n1 == 1:
                t1 = t + gen_va_exp(lamda_s1)
            lista_llegadas_sistema.append(t)

        # Caso en el que un cliente sale del primer sistema (o entra al segundo)
        elif evento_proximo == t1:
            t = t1
            n1 = n1-1
            if n1 == 0:
                t1 = math.inf
            else:
                t1 = t + gen_va_exp(lamda_s1)
            n2 = n2+1
            if n2 == 1:
                t2 = t + gen_va_exp(lamda_s2)
            numCliente = na - n1
            lista_llegadas_s2.append(t)

        # Caso en el que un cliente termina el segundo servicio y sale del sistema
        elif evento_proximo == t2:
            t = t2
            nd = nd+1
            n2 = n2-1

            # Caso en el que no hay mas clientes en el segundo sistema /en la cola)
            if n2 == 0:
                t2 = math.inf

            else:
                t2 = t + gen_va_exp(lamda_s2)
            numCliente = na - n1 - n2
            lista_salidas_sistema.append(t)
            tiempo_total_acumulado += (t - lista_llegadas_sistema[nd - 1])
        promedio_espera = tiempo_total_acumulado/nd
    return promedio_espera, lista_llegadas_sistema, lista_salidas_sistema
```

La función ***sim_servidores_serie(lamda_p, lamda_s1, lamda_s2, N)*** recibe como parámetros una cota máxima para la función de intensidad del proceso de Poisson (λ_p), la inversa del tiempo promedio de atención del servicio de Recepción (λ_{s1}), la inversa del tiempo promedio de atención del servicio de Validación (λ_{s2}), y un número máximo de clientes a atender (N).

Para simular nuestro problema, a partir de ahora cada vez que invoquemos a la función será con los siguientes argumentos.

$\lambda_p = 0.2$

Dado que nuestra función de intensidad es $\lambda(t) = 8 + 4 \sin(\pi t/12)$, como el $\sin(x)$ toma valores desde -1 a 1, y multiplicado por 4, los valores serían desde -4 a 4, si sumamos 8 a ambos límites del intervalo, tenemos que la función $\lambda(t)$ toma valores desde 4 a 12, por lo tanto, una cota máxima para la función $\lambda(t)$ es 12. Pero t está en horas, por lo que deberemos convertir la cota máxima dada en horas a minutos.

Por lo que **$\lambda_p = 12/60 = 0.2$**

$\lambda_{s1} = 1/4$

Dado que los tiempos de servicio en Recepción son variables aleatorias exponenciales con media de 4 minutos, entonces el parámetro λ de la exponencial será la inversa de la media, es decir, $1/4$.

$\lambda_{s2} = 1/6$

Dado que los tiempos de servicio en Validación son variables aleatorias exponenciales con media de 6 minutos, entonces el parámetro λ de una variable aleatoria exponencial será la inversa de la media, es decir, $1/6$.

$N = 10000$

Para poner un límite al sistema y detener la simulación, atenderemos a 10000 clientes.

Al finalizar la simulación, obtendremos los siguientes 3 resultados. El primero será un promedio de espera de los clientes en el sistema, el segundo es una lista con los tiempos de llegada de los clientes, y el tercero es una lista con los tiempos de salida de los clientes.

b) Utilizar el programa para estimar el tiempo promedio de permanencia de los clientes en el sistema. Construir intervalos de confianza para esta estimación.

Primero, usaremos montecarlos para estimar el tiempo promedio de permanencia de los clientes en el sistema.

Nuestro criterio de parada será cuando el **ancho** del intervalo de confianza del **95%** sea justo inferior a **0.5**

El intervalo de confianza para el valor esperado de una distribución estimado mediante la media muestral $X(n)$ a partir de una muestra de tamaño $n > 100$ está dado por

$$(X(n) - Z_{\alpha/2} * S(n) \div \sqrt{n}, X(n), X(n) + Z_{\alpha/2} * S(n) \div \sqrt{n})$$

$$\text{Semiancho del intervalo} = Z_{\alpha/2} * S(n) \div \sqrt{n}$$

$$\text{Ancho del intervalo} = 2 * Z_{\alpha/2} * S(n) \div \sqrt{n}$$

Por lo que, si queremos un intervalo menor que 0.5, con un 95% de confianza, el criterio de parada está dado por:

$$S(n) \div \sqrt{n} < L/2 * Z_{\alpha/2}$$

Y reemplazando $L = 0.5$, $Z_{\alpha/2} = 1.96$

$$S(n) \div \sqrt{n} < 0.5/2 * 1.96$$

Por lo que pararemos cuando

$$S(n) \div \sqrt{n} < 0.127551...$$

Implementación en python

```
def t_promedio_permanencia(L, N):
    media = sim_servidores_serie(0.2, 1/4, 1/6, N)[0]
    scua = 0.0
    n = 1
    d = L/(1.96**2)
    while (n < 100) or math.sqrt(scua/n) > d:
        n = n+1
        x = sim_servidores_serie(0.2, 1/4, 1/6, N)[0]
        media_ant = media
        media = media_ant + (x - media)/n
        scua = scua*(1-1/(n-1)) + n*((media_ant - media)**2)
        if n % 100 == 0:
            print(
                f"Iteración: {n} | Error actual: {math.sqrt(scua/n):.6f} | Objetivo: {d}"
            )
    intervalo = [media - 1.96 *
                 math.sqrt(scua/n), media + 1.96*math.sqrt(scua/n)]
    return media, math.sqrt(scua/n), n, intervalo
```

La función **t_promedio_permanencia(L,N)** recibe como parámetros una longitud de intervalo **L** , y un número de clientes máximo en el sistema a simular. Y devuelve tres resultados.

- La estimación del promedio de tiempo de clientes en el sistema.
- La desviación estándar del estimador.
- Las iteraciones necesarias para conseguir ese ancho de intervalo.
- Y los límites del intervalo.

Invocamos a la función **t_promedio_permanencia(L,N)** con los argumentos **L = 0.5**, y **N = 10.000**. Los resultados al ejecutar el programa son los siguientes.

```
Empezando estimacion del tiempo promedio de clientes en el sistema para 10000 clientes
Iteración: 100 | Error actual: 0.315241 | Objetivo: 0.12755102040816327
Iteración: 200 | Error actual: 0.215079 | Objetivo: 0.12755102040816327
Iteración: 300 | Error actual: 0.169141 | Objetivo: 0.12755102040816327
Iteración: 400 | Error actual: 0.142612 | Objetivo: 0.12755102040816327
Media estimada: 38.8096 minutos
Desviacion muestral del estimador: 0.1275
Numero de iteraciones requeridas: 491
Intervalo de confianza del 95%: [38.559692070039524, 39.05958512097404]
```

Podemos notar como el tiempo promedio de espera de los clientes es de 38.8 minutos, y que se necesitaron 491 iteraciones.

Si disminuimos el valor del intervalo, es decir **L**, el número de iteraciones aumentará significativamente, al punto de que con un intervalo de 0.1, las iteraciones necesarias serán mayores a 11.000

```
Iteración: 11700 | Error actual: 0.025545 | Objetivo: 0.025510204081632654
Media estimada: 38.6836 minutos
Desviacion muestral del estimador: 0.0255
Numero de iteraciones requeridas: 11727
Intervalo de confianza del 95%: [38.63359180050728, 38.733590930516264]
```

Otro dato a tener en cuenta es que a pesar de haber cambiado el tamaño del intervalo, la media estimada se mantiene en 38 minutos, y varían sus decimales, por lo que tomamos la decisión de mantener el intervalo de 0.5

- c) Considerar la atención de los primeros 10.000 clientes y registrar sus tiempos de permanencia en el sistema. Construir un histograma de los datos obtenidos y describir sus características principales.**

Implementación en python

```

# Muestra en pantalla un histograma del tiempo de espera de los clientes en el sistema
def imprimir_ej1c(N=10000):
    resultados = sim_servidores_serie(0.2, 1/4, 1/6, N)
    lista_t_llegadas = resultados[1]
    lista_t_salidas = resultados[2]
    lista_t_espera = obtener_tiempos_espera(lista_t_salidas, lista_t_llegadas)
    print("REALIZANDO EJERCICIO 1c")
    print("Imprimiendo diagrama")

    plt.figure(figsize=(11, 6))

    # density=True hace que el eje Y muestre la densidad de probabilidad
    plt.hist(lista_t_espera, bins='fd', color='skyblue',
             edgecolor='black')

    val_max = int(max(lista_t_espera)) + 20
    posiciones_x = list(range(0, val_max, 10))

    # Pasamos las posiciones de los numeros directamente como etiquetas
    plt.xticks(posiciones_x, posiciones_x, fontsize=10)

    plt.title(
        'Histograma de Tiempos de Permanencia en el Sistema (Primeros 10.000 clientes)', fontsize=14)
    plt.xlabel('Tiempo de permanencia en el sistema (Minutos)', fontsize=12)
    plt.ylabel('Cantidad de clientes (frecuencia)', fontsize=12)
    plt.grid(True, alpha=0.3)

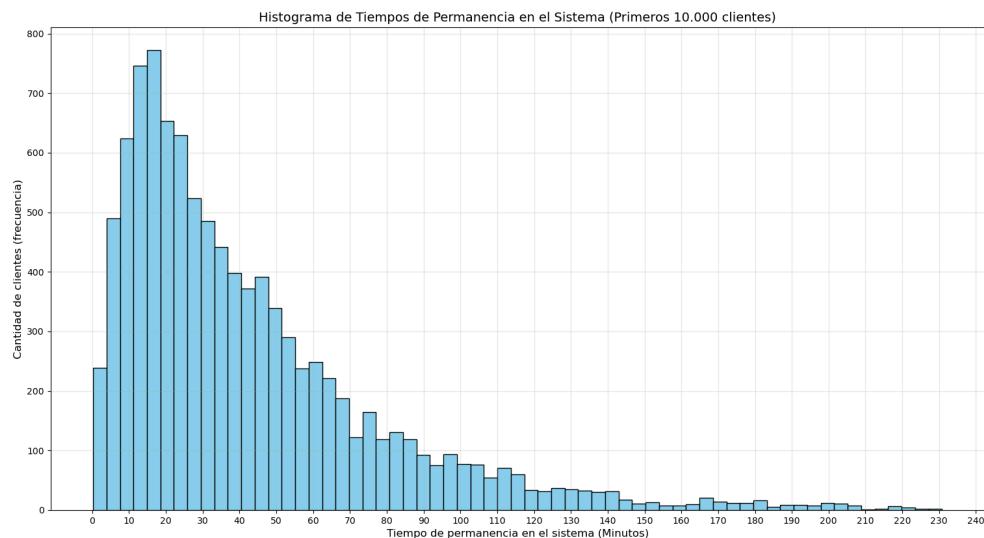
    # Mostrar el gráfico en pantalla
    plt.show()

```

La función `imprimir_ej1c()` muestra un histograma del tiempo de espera de los clientes en el sistema a partir de una muestra de 10.000 clientes.

Usamos la librería `matplotlib` para obtener el histograma por pantalla al ser invocada

A continuación un histograma de una muestra de 10.000, donde el eje X representa los minutos, y el eje Y las frecuencias, es decir, la cantidad de clientes que esperaron un intervalo minutos.



Podemos observar que la barra más alta, es de aproximadamente 780 clientes (menos de 800 pero más que 750) y esperaron entre 15 y 20 min. Luego hubo muy pocos clientes que permanecieron hasta 220 minutos en el sistema para ser atendidos.

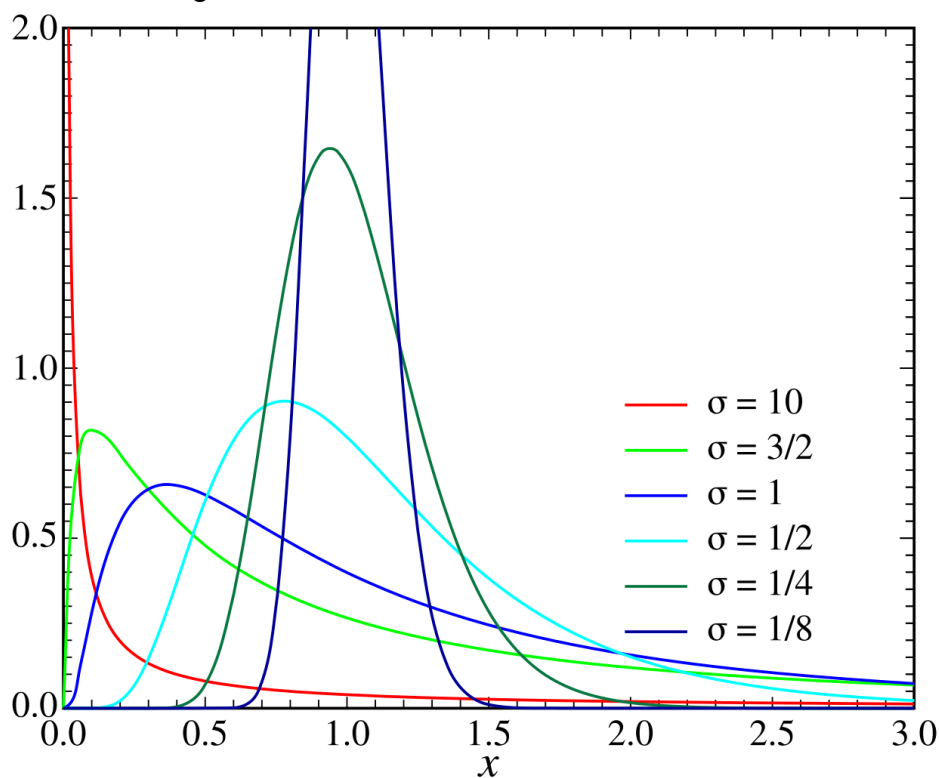
La mayoría de clientes permanecieron entre 10 y 35 minutos aproximadamente en el sistema.

Esa “cola” que se forma a la derecha del histograma, representa a los clientes que ingresaron durante los períodos de alta congestión y “saturaron” la cola de los servicios, por lo que tuvieron que permanecer más tiempo en el sistema.

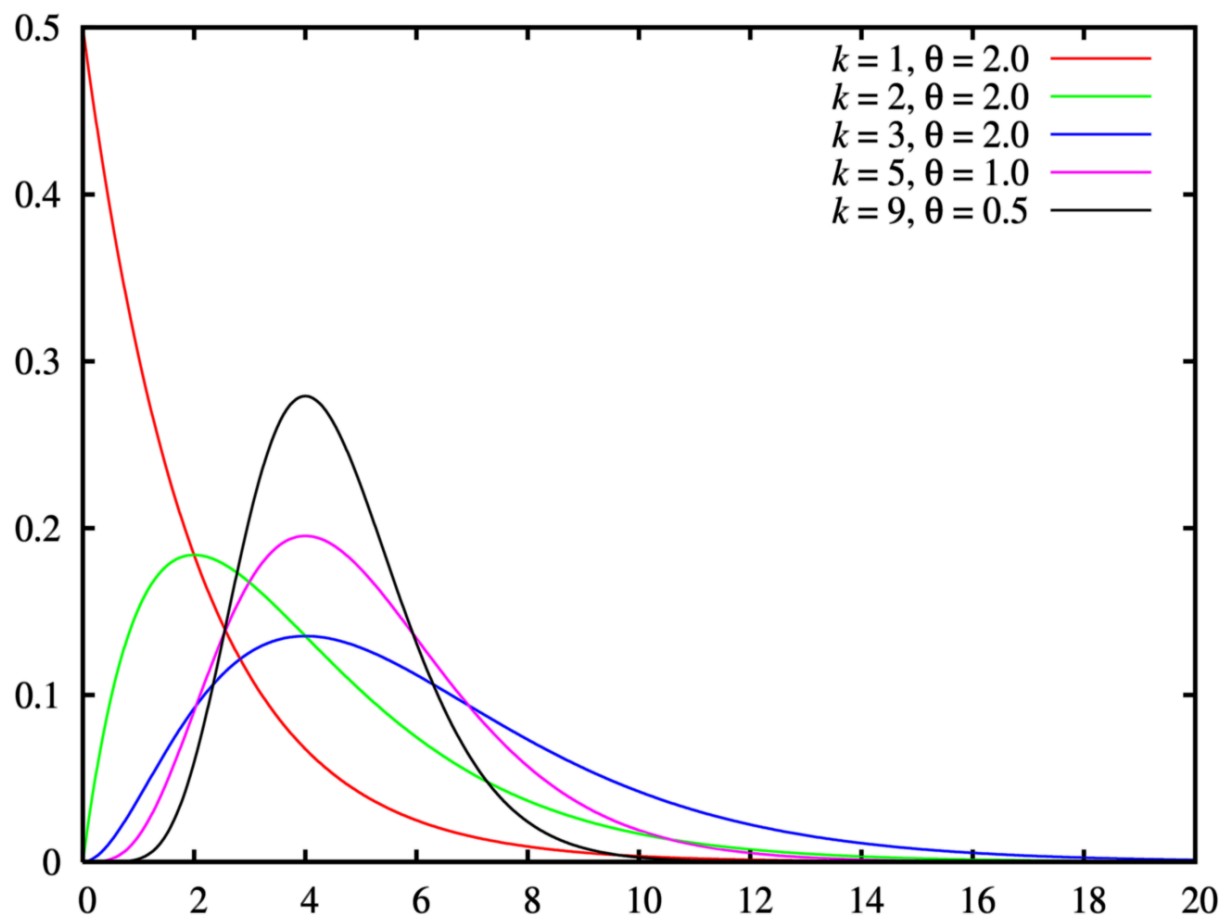
Ahora viendo este histograma surge la pregunta, **¿A qué función de distribución se parece?**

Proponemos dos distribuciones a las que se asemeja.

Distribución Log-Normal



Distribución Gamma



Realizaremos pruebas de bondad de ajuste χ^2 con un nivel de significación del 5 % para determinar si nuestra muestra proviene de una distribución gamma, o una log-normal.

Prueba de bondad de ajuste χ^2 con parámetros desconocidos con un nivel de significación del 5 % para el caso Gamma utilizando la prueba de Pearson con aproximación chi-cuadrada.

Planteamos nuestra hipótesis H_0 : La muestra proviene de una distribución Gamma (α, β) con parámetros desconocidos.

Como α y β son desconocidos, podemos estimarlos.

Si nuestra muestra proviene de una distribución Gamma, entonces, sabemos su esperanza y varianza.

$$E[X] = \alpha \cdot \beta$$

$$\text{Var}[X] = \alpha \cdot \beta^2$$

Si despejamos, podremos obtener alfa y beta de la siguiente manera.

$$\alpha = E[X]^2 / \text{Var}[X]$$

$$\beta = \text{Var}[X] / E[X]$$

Ahora, podemos estimar $E[X]$ como la media muestral $X(n)$, y la varianza $Var[X]$ como la varianza muestral S^2 .

Por lo que nos quedaría así:

$$\alpha = X(n)^2 / S^2$$

$$\beta = S^2 / X(n)$$

Luego, como nuestra muestra corresponde a datos continuos, no podemos contar las frecuencias y obtener los N_i (frecuencias observadas) para calcular nuestro estadístico. Por lo que tendremos que agrupar nuestros datos en ciertos intervalos, y usarlos como nuestros N_i .

Usaremos la regla de Sturges para calcular nuestros intervalos.

$$k = 1 + 3.332 \log(N)$$

donde k corresponde al número de intervalos, y N es el tamaño de la muestra.

Como tenemos una muestra de tamaño 10.000, entonces hacemos el cálculo.

$$k = 1 + 3.332 \log(10.000) = 15, \text{ por lo que tendremos 15 intervalos o } N_i$$

Ahora, obtenemos el rango, es decir la diferencia entre el valor máximo de nuestra muestra con respecto al valor mínimo.

$$\text{rango} = \max(\text{muestra}) - \min(\text{muestra})$$

Y para obtener el ancho que tendrá cada intervalo hacemos

$$\text{ancho intervalo} = \text{rango} / k$$

Conociendo el ancho del intervalo, podremos calcular los límites de cada intervalo.

Y determinar cuántas muestras cayeron en cada intervalo si están dentro de esos límites.

Para calcular las probabilidades teóricas, debemos tener en cuenta que si tenemos 15 intervalos, necesitaremos calcular la probabilidad p_i pero teniendo en cuenta los límites de cada intervalo.

$$P(\text{limite}_{i+1} < X < \text{limite}_i) = p_i$$

Ahora podremos calcular el estadístico de la siguiente manera.

$$T = \sum_{i=0}^{14} (N_i - N * p_i)^2 \div N * p_i$$

Luego, debemos ver la probabilidad $P(T \geq t)$, pero T tiene distribución Chi cuadrada con grados de libertad = $k-1-2$, ya que tenemos 2 parámetros desconocidos.

Entonces, $P(X_{12} \geq t) = p$, donde p es el p-valor, que en nuestro caso sería 0,05. Por lo que debemos calcular

$$P(X_{12} \geq t) = 0,05$$

Y determinar si nuestro t es menor a 0.05, entonces rechazaremos la hipótesis nula, en caso contrario, no la rechazamos.

Implementación en Python

```
def p_valor_gamma(N=10000):  
    # Corremos la simulacion una vez  
    resultados = sim_servidores_serie(0.2, 1/4, 1/6, N)  
  
    # Obtenemos una muestra de los tiempos de espera  
    muestra = obtener_tiempos_espera(resultados[2], resultados[1])  
    muestra_np = np.array(muestra)  
  
    # Calculamos la media y la varianza de la muestra  
    media_est = np.mean(muestra_np)  
    varianza_est = np.var(muestra_np, ddof=1)  
  
    # Estimamos alfa y beta  
    beta_est = varianza_est/media_est  
    alfa_est = media_est**2 / varianza_est  
  
    # Calculamos el numero de intervalos para agrupar nuestra muestra  
    k = calcular_numero_intervalos(N)  
  
    # Obtenemos una muestra agrupada, es decir los Ni  
    muestra_agrupada, limites = dar_muestra_intervalos(k, muestra_np)  
  
    # Calculamos las probabilidad teoricas para cada intervalo usando la distribucion Gamma con los parametros alfa y  
    # Es decir, calculamos los pi  
    prob = np.array([  
        stats.gamma.cdf(limites[i+1], a=alfa_est, loc=0, scale=beta_est) -  
        stats.gamma.cdf(limites[i], a=alfa_est, loc=0, scale=beta_est)  
        for i in range(k)  
    ])  
  
    # Obtenemos las frecuencias esperadas teoricas que obtuvimos a partir de la probabilidad teorica  
    frecuencias_esperadas = N*prob  
  
    # Realizamos la sumatoria de los (Ni-N*pi)**2 / N*pi  
    t = np.sum((muestra_agrupada-frecuencias_esperadas)**2 /  
        |         frecuencias_esperadas)  
  
    # Ahora calculamos el pvalor teorico usando la distribucion Chi-cuadrada  
    # Como tenemos 2 parametros desconocidos, 15 terminos en la sumatoria, los grados de libertad son 15-1-2  
    pvalor = stats.chi2.sf(t, df=k-2-1)  
    print(f"RESULTADOS DEL TEST CHI-CUADRADO")  
    print(f"Estadístico T calculado: {t}")  
    print(f"P-valor obtenido: {pvalor}")  
  
    if pvalor < 0.05: ...  
    else: ...  
  
    return pvalor, t
```

Veamos el resultado de una simulación.

```
RESULTADOS DEL TEST CHI-CUADRADO (GAMMA)  
Estadístico T calculado: 69.26226659782449  
P-valor obtenido: 4.4001455265945315e-10  
  
CONCLUSIÓN: RECHAZO H0 (P-valor < 0.05).  
Hay evidencia estadística suficiente para afirmar que los tiempos de espera NO siguen una distribución Gamma con parámetros alfa = 1.7418 y beta = 22.0730
```

El p-valor obtenido es muy pequeño, es decir $p\text{-valor} < 0.05$, y tenemos que rechazar la hipótesis Nula. Y por lo tanto, los datos no provienen de una distribución Gamma de parámetros alfa y beta estimados.

Prueba de bondad de ajuste χ^2 con parámetros desconocidos con un nivel de significación del 5 % para el caso Log-Normal utilizando la prueba de Pearson con aproximación chi-cuadrada.

Planteamos nuestra hipótesis H_0 : La muestra proviene de una distribución Lognormal con parámetros (μ, σ) desconocidos.

Al igual que al plantear un test de bondad chi cuadrado para el caso Gamma, los pasos son los mismos, salvo que tendremos que estimar los parámetros (μ, σ) . Y en el paso de calcular los p_i teóricos, en vez de usar una distribución Gamma, usaremos una distribución Log-normal. El resto de los pasos, son similares, y otros iguales. Por lo que procedemos a implementarlo en Python.

Implementación en python

```
def p_valor_lognormal(N=10000):
    # Corremos la simulación una vez
    resultados = sim_servidores_serie(0.2, 1/4, 1/6, N)

    # Obtenemos una muestra de los tiempos de espera
    muestra = obtener_tiempos_espera(resultados[2], resultados[1])
    muestra_np = np.array(muestra)

    # Aplicamos logaritmo natural para estimar los parametros de la Log-normal
    datos_log = np.log(muestra_np)
    mu_est = np.mean(datos_log)
    sigma_est = np.std(datos_log, ddof=1)

    # Calculamos el numero de intervalos para agrupar nuestra muestra
    k = calcular_numero_intervalos(N)

    # Obtenemos una muestra agrupada, es decir los Ni
    muestra_agrupada, limites = dar_muestra_intervalos(k, muestra_np)

    # Calculamos las probabilidad teoricas para cada intervalo usando la distribucion Log-normal
    prob = np.array([
        stats.lognorm.cdf(limites[i+1], s=sigma_est, loc=0, scale=np.exp(mu_est)) -
        stats.lognorm.cdf(limites[i], s=sigma_est, loc=0, scale=np.exp(mu_est))
        for i in range(k)
    ])

    # Obtenemos las frecuencias esperadas teoricas que obtuvimos a partir de la probabilidad teorica
    frecuencias_esperadas = N * prob

    # Realizamos la sumatoria de los  $(N_i - N \cdot p_i)^2 / N \cdot p_i$ 
    t = np.sum((muestra_agrupada - frecuencias_esperadas)
               ** 2 / frecuencias_esperadas)

    # Ahora calculamos el pvalor teorico usando la distribucion Chi-cuadrada
    # Al igual que antes, estimamos 2 parametros (mu y sigma), por lo que df = k - 2 - 1
    grados_libertad = k - 2 - 1
    pvalor = stats.chi2.sf(t, df=grados_libertad)

    print(f"RESULTADOS DEL TEST CHI-CUADRADO (LOG-NORMAL)")
    print(f"Estadístico T calculado: {t}")
    print(f"P-valor obtenido: {pvalor}")

> if pvalor < 0.05: ...
> else: ...

return pvalor, t
```

```
RESULTADOS DEL TEST CHI-CUADRADO (LOG-NORMAL)
Estadístico T calculado: 128.05794621743564
P-valor obtenido: 1.513498293187061e-21
```

```
CONCLUSIÓN: RECHAZO H0 (P-valor < 0.05).
Hay evidencia estadística suficiente para afirmar que los tiempos de espera NO siguen una distribución Log-normal con parámetros  $\mu = 3.2907$  y  $\sigma = 0.8707$ .
```

El p-valor obtenido es muy pequeño, es decir $p\text{-valor} < 0.05$, y tenemos que rechazar la hipotética Nula. Y por lo tanto, los datos no provienen de una distribución Log-normal de parámetros (μ, σ) desconocidos y estimados.

CONCLUSIONES

Dado que en ambas pruebas de bondad de ajuste χ^2 con un nivel de significación del 5% rechazamos la hipótesis nula (en un caso para la distribución Gamma, y en otro caso para la Log-Normal), podemos intuir que una causa posible es la estimación de sus parámetros (α, β) para Gamma, y (μ, σ) para Log-normal. Otra posible razón es el tamaño de la muestra 10.000, en estos casos, el estadístico se vuelve muy “estricto” y detecta mejor las diferencias e imperfecciones, rechazando nuestra hipótesis.

Además, el número de intervalos K calculado es muy chico, por lo que el tamaño de cada intervalo resulta muy grande; esto provoca que los datos se suavicen y pierdan la forma de su verdadera distribución, haciendo que el estadístico penalice el modelo y rechace H_0 .

Actividad 2

Consigna:

Se solicita separar en dos tipo de clientes la etapa de validación:

- Clientes Prioritarios
- Clientes Normales

Donde la probabilidad de ser prioritario es 0,20 (independientemente del resto).

La etapa de Recepción funciona de igual manera como en la Actividad 1.

La nueva metodología de la Validación es la siguiente:

- Si hay clientes prioritarios esperando, se atiende al primero de ellos.
- Una vez iniciado un servicio, éste no puede ser interrumpido
- Los clientes normales sólo son atendidos cuando no quedan clientes prioritarios en espera.

Incisos:

a) Adaptar el programa desarrollado en el ejercicio anterior para incorporar clientes prioritarios.

b) Estimar el tiempo promedio de permanencia en el sistema para clientes normales y para clientes prioritarios. Construir intervalos de confianza del 95% para ambas cantidades.

c) Construir histogramas de los tiempos de permanencia para ambos tipos de clientes y describir las diferencias observadas: media, mediana, percentiles, probabilidad de esperar más de x minutos.

d) Compare los resultados obtenidos con los del Ejercicio 1 y discuta el impacto de la política de prioridades sobre ambos grupos de clientes.

2.1. Item A | Adaptación

Lo que se hizo para la adaptación a los dos tipos de clientes fue agregar 2 listas auxiliares para representar sub-colas de espera de normales y prioritarios. Luego de determinar si el cliente es prioritario o no mediante una Bernoulli, se almacena su número de cliente en la lista correspondiente. Al ser atendido un cliente se almacena su tipo de prioridad en una variable auxiliar para tener una referencia al final de la Validación.

A continuación se muestra la implementación en uno de los casos del sistema:

```
#Caso en el que un cliente sale del primer sistema (o entra al segundo)
elif evento_proximo == t1:
    t = t1
    n1 = n1-1
    numCliente = na - n1
    if n1 == 0:
        t1 = math.inf
    else:
        t1 = t + gen_va_exp(lamda_s1)
    n2 = n2+1
    #Comprobacion de si el cliente que entra al segundo sistema es prioritario o no
    es_prioritario = gen_va_bernoulli(0.2)
    if es_prioritario:
        p2.append(numCliente)
        if len(p2) == 1:
            t2 = t + gen_va_exp(lamda_s2)
            #Se asigna el tipo de cliente que se encuentra en el segundo sistema para luego calcular su tiempo de espera total al salir del sistema
            cliente_en_s2 = "prioritario"
        else:
            l2.append(numCliente)
            if len(l2) == 1 and len(p2) == 0:
                t2 = t + gen_va_exp(lamda_s2)
                cliente_en_s2 = "normal"

    lista_llegadas_s2.append(t)
```

2.2. Item B | Estimación de Tiempos

Para la estimación de permanencia en el sistema de ambos tipos de clientes se implementó mediante el uso de 2 listas de tiempos totales para cada tipo de cliente. En el caso que un cliente saliera de la Validación, mediante el uso de su número de cliente como índice se busca su tiempo de llegada al sistema y se calcula su tiempo Total de permanencia, el mismo es guardado en la lista de tiempos del tipo correspondiente.

Las listas con los tiempos de cada Tipo se utilizan para calcular los promedios Totales (ambos tipo de clientes en conjunto) y los parciales (ambos tipo de clientes por separado). Además estas listas son devueltas al final del algoritmo implementado para su posterior análisis.

A continuación una imagen de la implementación para estimar tiempos:

```
#Caso en el que un cliente termina el segundo servicio y sale del sistema
elif evento_proximo == t2:
    t = t2
    nd = nd+1
    n2 = n2-1
    #Se calcula el tiempo total en sistema para ambos tipos de clientes.
    #y se elimina al cliente que salio del sistema de la lista correspondiente (prioritarios o normales)
    if cliente_en_s2 == "prioritario":
        tiempo_total_prioritarios.append(t - lista_llegadas_sistema[p2[0] - 1])
        p2.pop(0)
    else:
        tiempo_total_normales.append(t - lista_llegadas_sistema[l2[0] - 1])
        l2.pop(0)
```

A continuación una muestra del cálculo de promedios:

```
# Calculo de los promedios de espera para ambos tipos de clientes y en general
promedio_espera = sum(tiempo_total_prioritarios + tiempo_total_normales) / nd if nd > 0 else 0

promedio_espera_prioritarios = sum(tiempo_total_prioritarios)/len(tiempo_total_prioritarios) if len(tiempo_total_prioritarios) > 0 else 0

promedio_espera_normales = sum(tiempo_total_normales)/len(tiempo_total_normales) if len(tiempo_total_normales) > 0 else 0
```

Para la construcción de intervalos de confianza del 95% para ambos datos se implementó el algoritmo de simulaciones hasta cumplir con las siguientes condiciones:

- $n \geq 100$ réplicas mínimas
- $\sqrt{S^2/n} \leq d$ donde $d = \frac{L}{2 \cdot z_{\alpha/2}}$

Donde d representa el Semiancho máximo del intervalo de confianza.

Esta condición se replicó para cada dato, buscando intervalos de confianza del 95% para el promedio de espera de normales, prioritarios y el total.

Implementación en Python

```
def t_promedio_permanencia(L,N):
    total, prio, normal, __, _ = sim_servidores_serie(0.2,1/4,1/6,N)

    media_total = total
    media_prio = prio
    media_normal = normal

    scud_total = 0.0
    scud_prio = 0.0
    scud_normal = 0.0
    n = 1
    d = L/(1.96*2)
    while (n < 100) or (math.sqrt(scud_total/n) > d or
                      math.sqrt(scud_prio/n) > d or
                      math.sqrt(scud_normal/n) > d):

        n = n+1
        total, prio, norm, __, _ = sim_servidores_serie(0.2, 1/4, 1/6, N)

        media_ant_total, media_ant_prio, media_ant_normal = media_total, media_prio, media_normal

        # Actualizacion de las medias con la formula de Welford para la media muestral
        media_total += (total - media_total) / n
        media_prio += (prio - media_prio) / n
        media_normal += (norm - media_normal) / n

        # Actualizacion de las scuds con la formula de Welford para varianza muestral
        scud_total = scud_total * (1 - 1/(n-1)) + n * (media_ant_total - media_total)**2
        scud_prio = scud_prio * (1 - 1/(n-1)) + n * (media_ant_prio - media_prio)**2
        scud_normal = scud_normal * (1 - 1/(n-1)) + n * (media_ant_normal - media_normal)**2

    # Calculo de Los semianchos de Los intervalos de confianza para cada tipo de cliente y en general
    semi_total = 1.96 * math.sqrt(scud_total/n)
    semi_prio = 1.96 * math.sqrt(scud_prio/n)
    semi_normal = 1.96 * math.sqrt(scud_normal / n)

    # Imprimir resultados cada 1000 iteraciones
    if n % 1000 == 0:
        print(f"Iteración: {n} | Error actual: {math.sqrt(scud_total/n):.6f} | Objetivo: {d}")

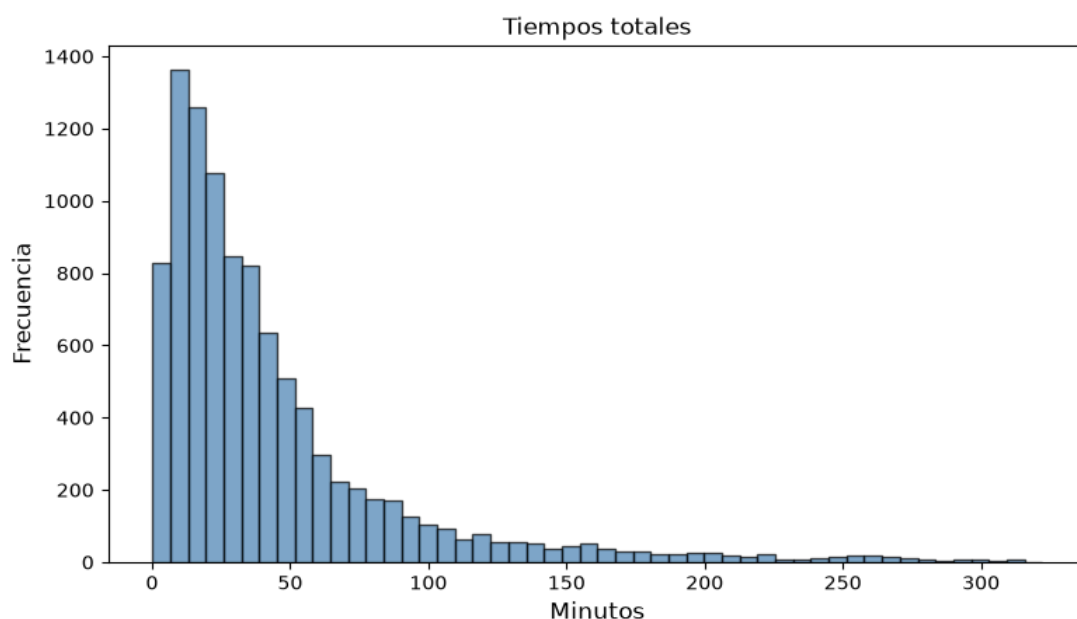
    # Devolver los resultados en un formato legible, incluyendo el número de iteraciones,
    # las medias y los intervalos de confianza para cada tipo de cliente y en general
    return {
        "Nº iteraciones": n,
        "Total": (media_total, [media_total - semi_total, media_total + semi_total]),
        "Prioritarios": (media_prio, [media_prio - semi_prio, media_prio + semi_prio]),
        "Normales": (media_normal, [media_normal - semi_normal, media_normal + semi_normal])
    }
```

2.3. Item C & D | Análisis de Datos y Comparacion

En este inciso analizaremos los datos obtenidos luego de incluir al algoritmo utilizado el sistema de clientes prioritarios.

Primeramente analizaremos los tiempos de permanencia en sistema de los clientes mediante Histogramas.

A continuación el gráfico de los datos obtenidos de permanencia para Todos los clientes:

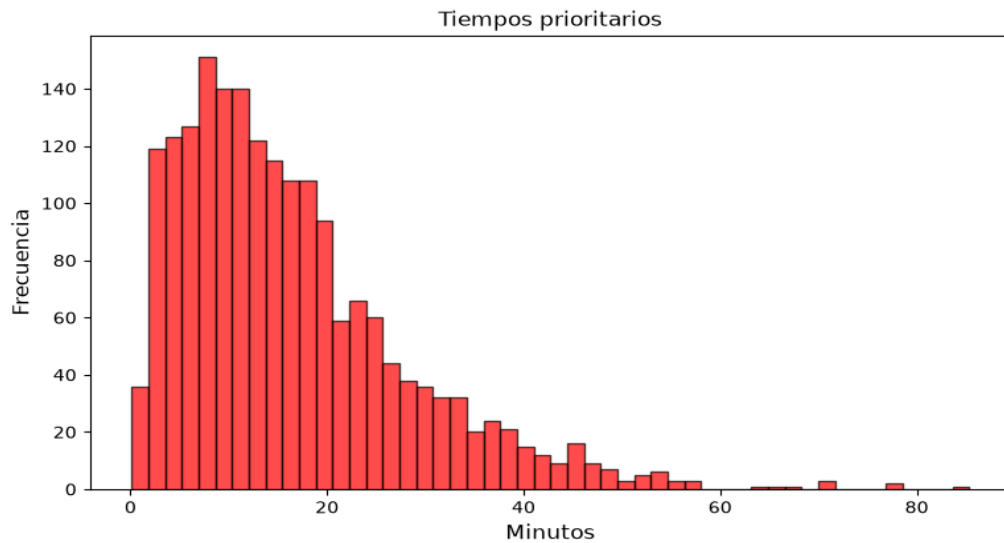


Lo primero que podemos notar a comparación de los datos obtenidos en la Actividad 1 es que muchos clientes esperan notablemente mucho más tiempo llegando hasta 300 minutos de espera.

Esto se explica por muchos clientes normales los cuales debieron esperar un mayor tiempo debido a los prioritarios y normales anteriores.

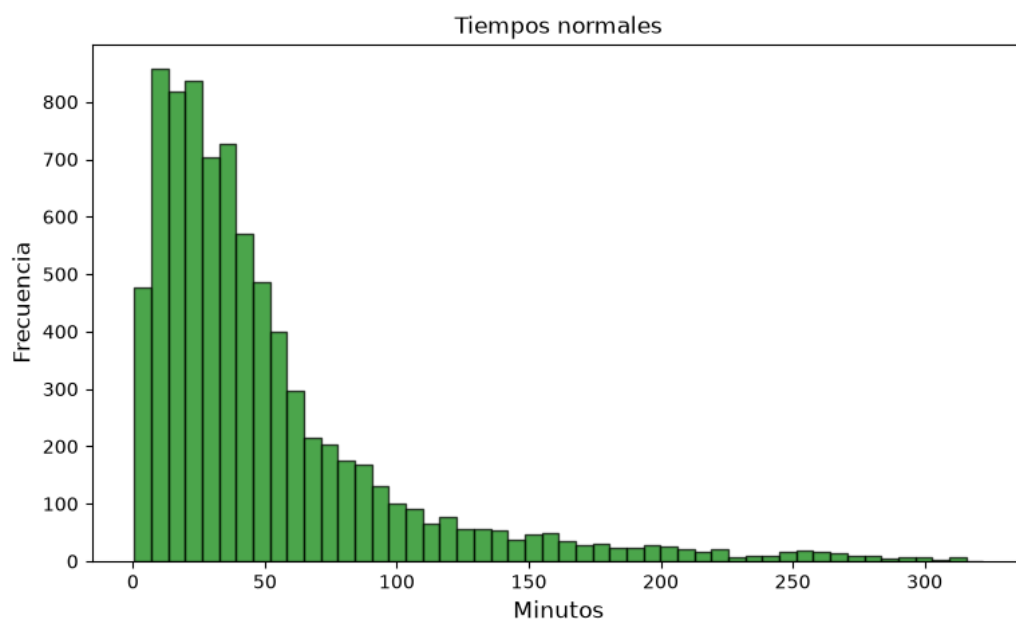
Luego los picos de frecuencia en los minutos iniciales es mucho mayor en comparación con el gráfico de la anterior Actividad, esto lógicamente se debe a los clientes prioritarios los cuales son atendidos en la brevedad la gran mayoría. Lo cual termina afectando a la gran cantidad de clientes Normales que se acumulan en la cola de la etapa de Validación.

A continuación gráfico con datos de permanencia para clientes Prioritarios:



Notemos como se mencionó anteriormente, la gran mayoría de los clientes Prioritarios esperan menos de 40 minutos. Luego otros pocos que han esperado un tiempo mucho mayor, incluso el doble se debe a la espera de otros Prioritarios y a la espera de clientes Normales siendo atendidos (por el hecho de que un servicio no puede ser interrumpido).

A continuación gráfico con datos de permanencia para clientes Normales:



En este gráfico podemos observar y confirmar como los clientes Normales son altamente perjudicados en los primeros 60 minutos en correlación con el resto de gráficos.

También corroboramos que la gran mayoría de clientes que esperan cantidades considerables de tiempo son Normales.

Diferencia de Datos

Ahora analizaremos la diferencia entre clientes Normales y Prioritarios desde el punto de vista de los datos obtenidos teniendo en cuenta Media, Mediana, Percentiles y probabilidades de esperar más de X minutos.

Primeramente el cálculo de la Media utilizado de forma teórica con los tiempos resultantes:

$$\overline{X}(n) = \frac{1}{n} (X_1 + X_2 + \dots + X_n).$$

(en el código implementado se utilizó la librería NumPy para los cálculos de las mismas)

Luego de la ejecución de las simulaciones mediante el método de Monte Carlo nuestros resultados fueron:

```
Medias e intervalos de confianza para el tiempo promedio de permanencia en el sistema:

Resultados para L = 0.5 minutos y N = 10000 clientes:

{'N iteraciones': 708, 'Total': (38.71219380792963, [38.5101577988496, 38.91422981700966]), 'Prioritarios': (15.858219518536103, [15.828499131131075, 15.887939905941131]), 'Normales': (44.41926880784776, [44.169298265253566, 44.66923935044195])}
```

Notemos que a diferencia de la Actividad anterior en muy considerable la diferencia de iteraciones necesarias para lograr las condiciones solicitadas, llegando casi al doble.

Otra cosa que podemos identificar es que la media total se mantiene prácticamente igual en 38,71 minutos.

Por otro lado, el tiempo medio esperado por clientes prioritarios y normales nos confirman los análisis realizados en base a los gráficos anteriormente mencionados.

A continuación veremos los datos separado por Tipo de cliente para una muestra de 10000 clientes:

```
--- Normales ---
Media:      42.4745
Mediana:    32.1514
P25:        16.7301
P75:        58.1762
P90:        90.3464
```

```
--- Prioritarios ---
Media:      16.1326
Mediana:    13.6034
P25:        7.5023
P75:        22.1544
P90:        31.3534
```

Podemos observar como en los clientes Normales la media es bastante mayor que la mediana, lo que nos indica una distribución asimétrica hacia la derecha. Luego por los percentiles podemos ver que el 50% de ellos espera entre 16 y 58 minutos, lo cual es un rango muy amplio lo cual nos refleja una gran variabilidad. Luego el p90 nos indica que el 10% más demorado de los Normales suelen esperar más de 90 minutos.

En el caso de los clientes Prioritarios vemos que están mucho más cercanas a la Media y la Mediana, lo cual nos indica que está mucho más simétrica su distribución.

Luego el 50% central por los percentiles P25 y P75 nos indican que esperan entre 7 y 22 minutos, lo cual nos confirma menor variabilidad.

En cuanto al P90 nos indica que el 10% de los más demorados espera cerca de 32 minutos o más, una diferencia realmente amplia con los clientes Normales.

Ahora para estimar la probabilidad de que cada tipo de cliente supere un X minutos se planteó $X = 10 / 20 / 30 / 60$.

P(W > 10 min)	- Prioritarios: 0.6547	Normal: 0.8946
P(W > 20 min)	- Prioritarios: 0.2891	Normal: 0.7316
P(W > 30 min)	- Prioritarios: 0.1177	Normal: 0.5823
P(W > 60 min)	- Prioritarios: 0.0094	Normal: 0.2810

Podemos notar como solo el 28% de los clientes Prioritarios esperan más de 20 minutos contra un 73% de los clientes Normales, y la amplia diferencia entre ambos tipos de clientes se mantiene a lo largo del tiempo, llegando a esperar hasta más de 60 minutos el 28% de los Normales y tan solo 0,9% de los prioritarios.

Conclusión

La elección de implementar prioridades depende enteramente del objetivo del sistema. Si el objetivo es garantizar atención rápida a un subconjunto crítico de usuarios, la política es efectiva. Sin embargo, si el objetivo es equidad o minimizar la espera máxima, la política resulta perjudicial para la mayoría, ya que el 80% de los clientes normales ve empeorada su experiencia respecto al sistema sin prioridades.

